



K8s Introduction

Containers – Why you need Containers

Let's assume I am building a project projects, the requirement is to setup an end to end stack including various different technologies like a Web Server using NodeJS and a database such as MongoDB/ CouchDB , messaging system like Redis and an orchestration tool like Ansible .

I have a lot of issues developing this application with all these different components.

First, their compatibility with the underlying OS. You have to ensure that all these different services were compatible with the version of the OS you are planning to use. There have been times when certain version of these services were not compatible with the OS, and I have had to go back and look for another OS that was compatible with all of these different services.

Secondly, I have to check the compatibility between these services and the libraries and dependencies on the OS. We have had issues were one service requires one version of a dependent library whereas another service required another version.

Containers – Why you need Containers

Everytime we had a new developer on board, we found it really difficult to setup a new environment. The new developers had to follow a large set of instructions and run 100s of commands to finally setup their environments.

They had to make sure they were using the right Operating System, the right versions of each of these components and each developer had to set all that up by himself each time.

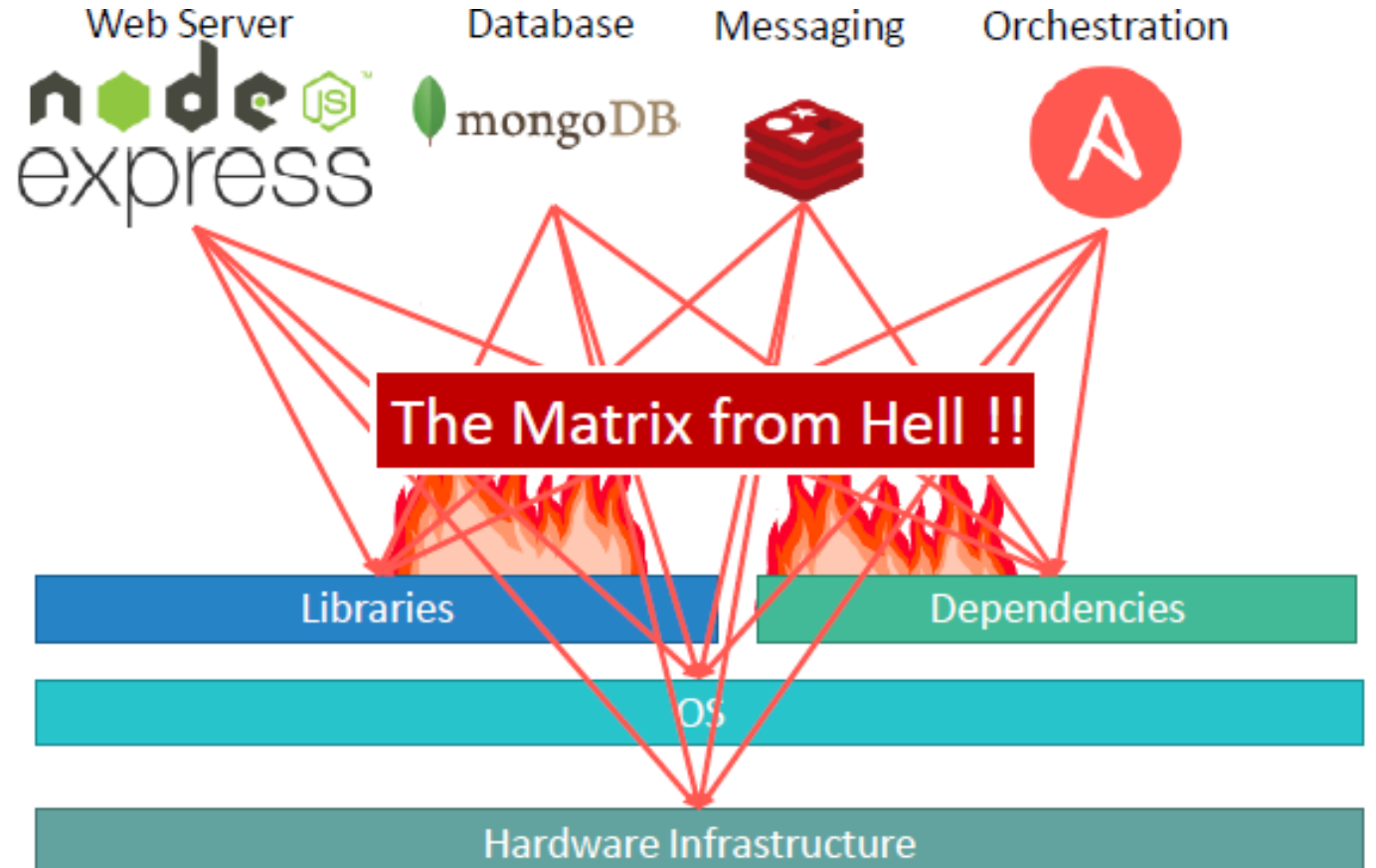
We also had different development test and production environments. One developer may be comfortable using one OS, and the others may be using another one and so we couldn't guarantee the application that we were building would run the same way in different environments. And So all of this made our life in developing,

Containers – Why you need Containers

Compatibility/Dependency

Long setup time

Different Dev/Test/Prod environments



Containers – Why you need Containers

So I needed something that could help us with the compatibility issue. And something that will allow us to modify or change these components without affecting the other components and even modify the underlying operating systems as required. And that search landed me on Docker.

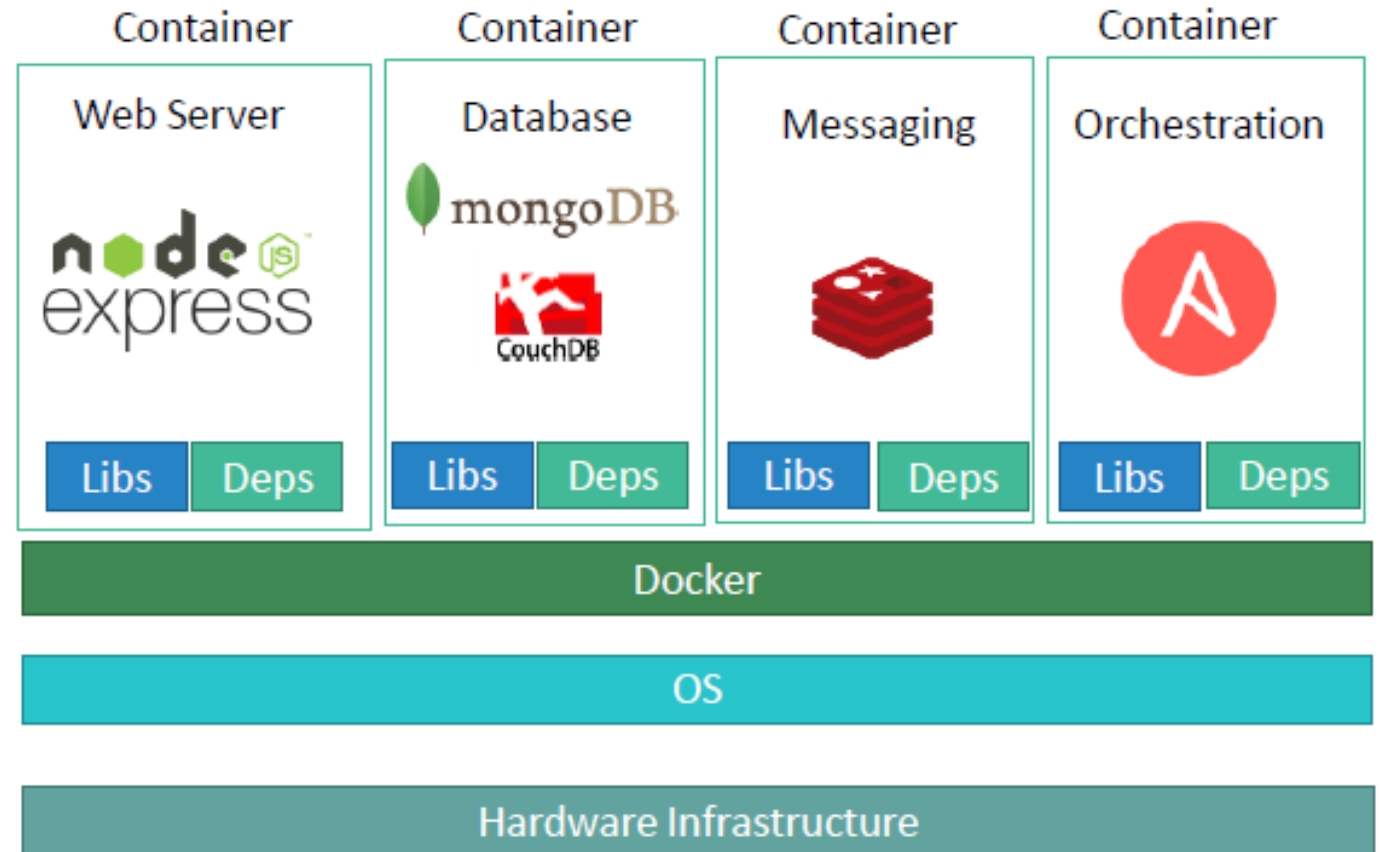
With Docker I was able to run each component in a separate container with its own libraries and its own dependencies. All on the same VM and the OS, but within separate environments or containers. We just had to build the docker configuration once, and all our developers could now get started with a simple “ docker run” command. Irrespective of what underlying OS they run, all they needed to do was to make sure they had Docker installed on their systems.

So what are containers? Containers are completely isolated environments, as in they can have their own processes or services, their own network interfaces, their own mounts, just like Virtual machines, except that they all share the same OS kernel.

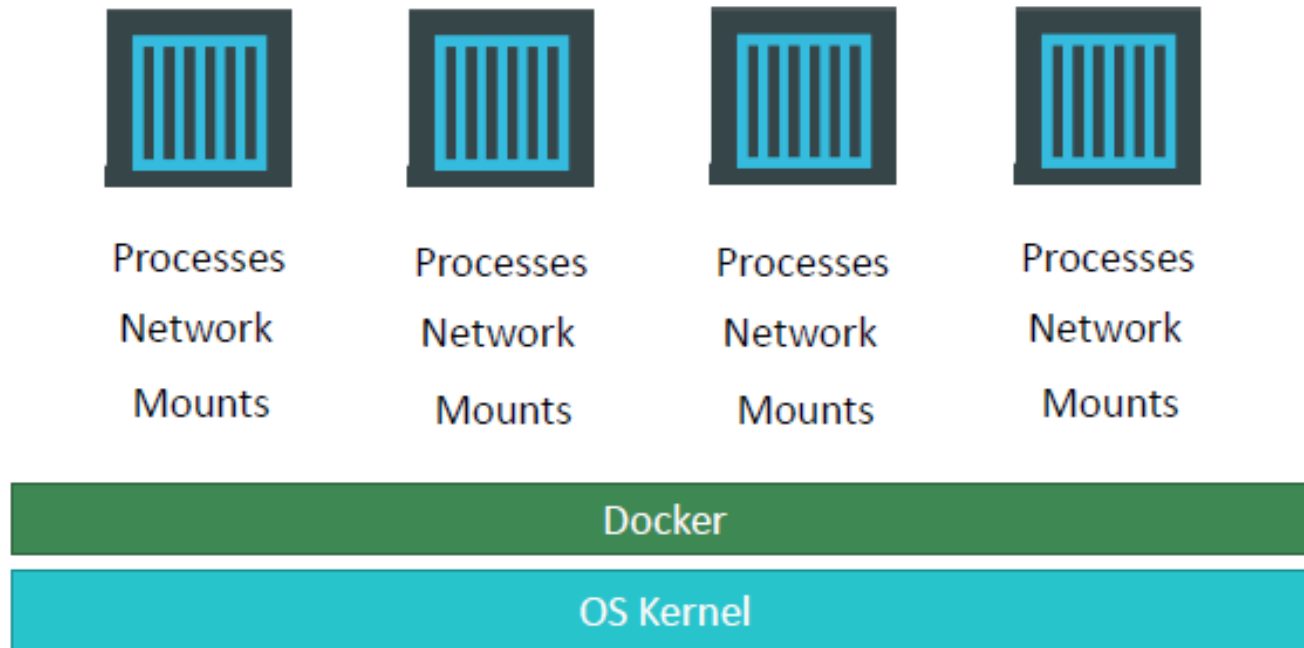
Containers – a solution

Containerize Applications

Run each service with its own dependencies in separate containers



Containers – Why you need Containers



Containers – Why you need Containers

To understand how Docker works let us revisit some basics concepts of Operating Systems first. If you look at operating systems like Ubuntu, Fedora, Suse or Centos they all consist of two things. An OS Kernel and a set of software.

The OS Kernel is responsible for interacting with the underlying hardware. While the OS kernel remains the same which is Linux in this case, it's the software above it that make these Operating Systems different. This software may consist of a different User Interface, drivers, compilers, File managers, developer tools etc.

SO you have a common Linux Kernel shared across all OSES and some custom softwares that differentiate Operating systems from each other.

Containers – Why you need Containers

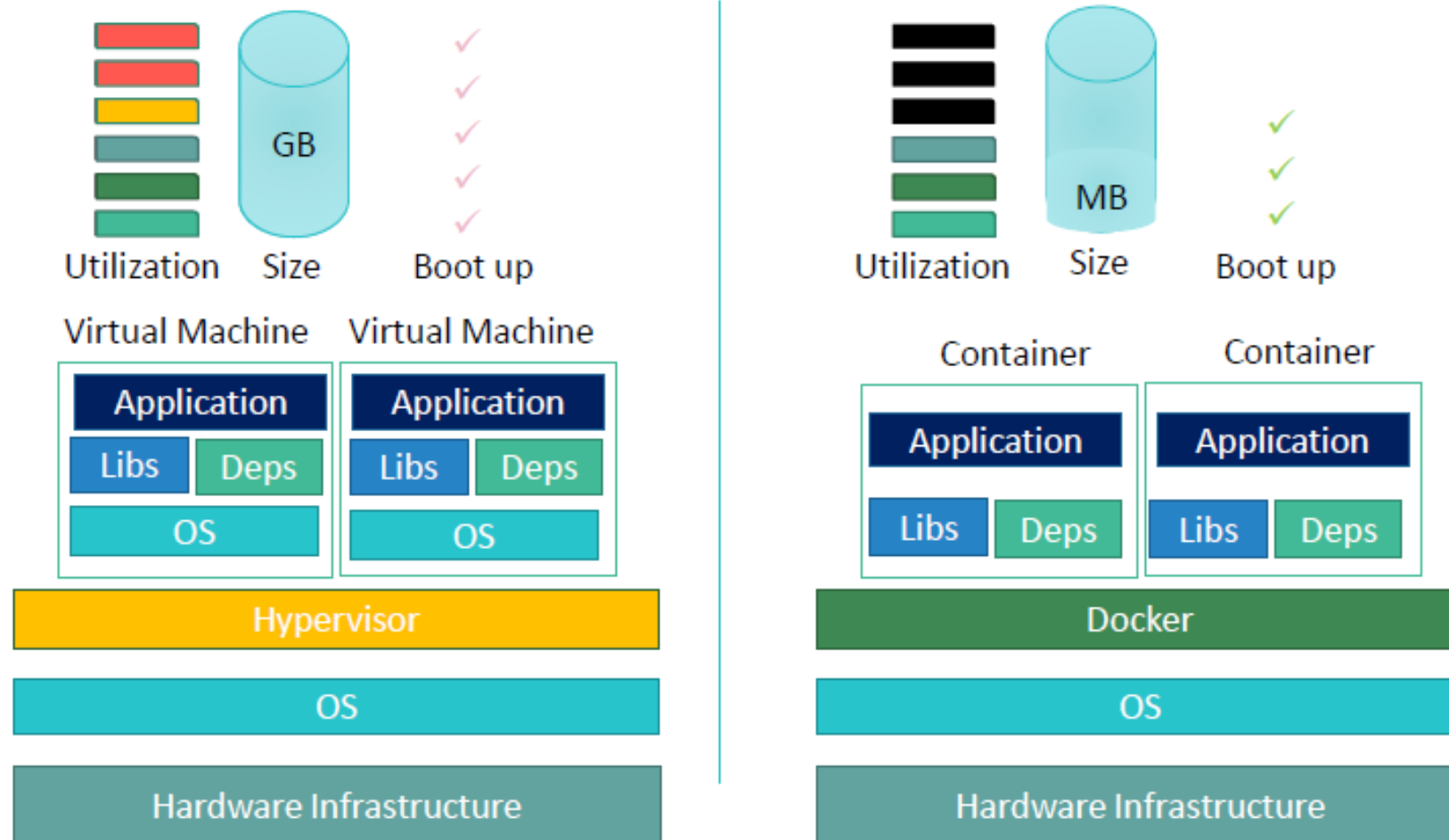
We said earlier that Docker containers share the underlying kernel. What does that actually mean sharing the kernel? Let's say we have a system with an Ubuntu OS with Docker installed on it. Docker can run any flavor of OS on top of it as long as they are all based on the same kernel in this case Linux. If the underlying OS is Ubuntu, docker can run a container based on another distribution like debian , fedora, suse or centos.

Each docker container only has the additional software, that we just talked about in the previous slide, that makes these operating systems different and docker utilizes the underlying kernel of the Docker host which works with all Oses above.

So what is an OS that do not share the same kernel as these? Windows ! And so you wont be able to run a windows based container on a Docker host with Linux OS on it. For that you would require docker on a windows server.

You might ask isn't that a disadvantage then? Not being able to run another kernel on the OS? The answer is No! Because unlike hypervisors, Docker is not meant to virtualize and run different Operating systems and kernels on the same hardware. The main purpose of Docker is to containerize applications and to ship them and run them.

Containers vs. Virtual Machines



Containers vs. Virtual Machines

As you can see on the right, in case of Docker, we have the underlying hardware infrastructure, then the OS, and Docker installed on the OS. Docker then manages the containers that run with libraries and dependencies alone.

In case of a Virtual Machine, we have the OS on the underlying hardware, then the Hypervisor like a ESX or virtualization of some kind and then the virtual machines. As you can see each virtual machine has its own OS inside it, then the dependencies and then the application.

This overhead causes higher utilization of underlying resources as there are multiple virtual operating systems and kernel running. The virtual machines also consume higher disk space as each VM is heavy and is usually in Giga Bytes in size, whereas docker containers are lightweight and are usually in Mega Bytes in size.

This allows docker containers to boot up faster, usually in a matter of seconds whereas VMs we know takes minutes to boot up as it needs to bootup the entire OS.

Containers vs. Image

An image is a package or a template, just like a VM template that you might have worked with in the virtualization world. It is used to create one or more containers.

Containers are running instances off images that are isolated and have their own environments and set of processes

As we have seen before a lot of products have been dockerized already. In case you cannot find what you are looking for you could create an image yourself and push it to the Docker hub repository making it available for public.



Docker Image

Package
Template
Plan



Docker Container #1



Docker Container #2



Docker Container #3

What is Dockerfile

A Dockerfile is a plain text file that contains a series of instructions used to build a Docker image. Each line in a Dockerfile represents a step in the image-building process.

Key Components of a Dockerfile:

1. **Base Image:** The starting point for your Docker image. For example, if you're building a Python application, you might start with `python:3.9` as your base image.
2. **Application Code and Dependencies:** The code is added to the image, and dependencies are installed to ensure the application runs correctly.
3. **Commands and Configurations:** Instructions to execute commands, set environment variables, and expose ports.

Dockerfile Syntax

A Dockerfile contains simple instructions, where each instruction performs a specific action. The syntax is generally:

INSTRUCTION arguments

For example:

FROM ubuntu:20.04

COPY . /app

RUN apt-get update && apt-get install -y python3

CMD ["python3", "/app/app.py"]

Key points:

- Instructions like FROM, COPY, RUN, and CMD are case-sensitive and written in uppercase.
- Each instruction creates a new layer in the Docker image.

Common Instructions

FROM

Specifies the base image for your build. Example:

FROM python:3.9

A Dockerfile must start with a FROM instruction, except in multi-stage builds.

COPY

Copies files or directories from the host system into the container. Example:

COPY requirements.txt /app/

RUN

Executes commands during the build process. Often used to install packages.

RUN apt-get update && apt-get install -y curl

Common Instructions

CMD

Specifies the default command to run when the container starts.

CMD ["python3", "app.py"]

WORKDIR

Sets the working directory inside the container.

WORKDIR /usr/src/app

EXPOSE

Documents the port the container listens on.

EXPOSE 8080

Dockerfile example

Use official Python image as base

FROM python:3.11-slim

Set working directory inside container

WORKDIR /app

Copy requirements file first (better for caching)

COPY requirements.txt .

Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

Copy the rest of the application code

COPY . .

Expose port (Flask default example)

EXPOSE 5000

Run the application

CMD ["flask", "run"]

Containers Orchestration

But what next? How do you run it in production? What if your application relies on other containers such as database or messaging services or other backend services? What if the number of users increase and you need to scale your application? You would also like to scale down when the load decreases.

To enable these functionalities you need an underlying platform with a set of resources. The platform needs to orchestrate the connectivity between the containers and automatically scale up or down based on the load.

This whole process of automatically deploying and managing containers is known as Container Orchestration.

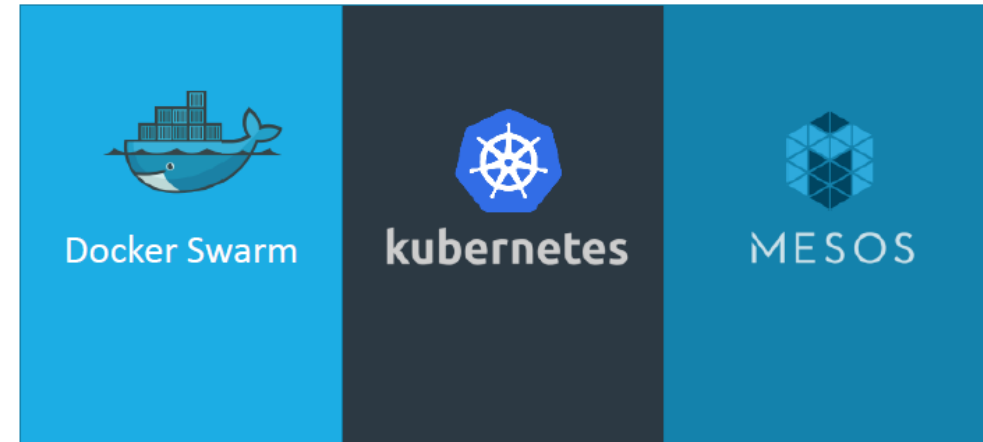
Containers Orchestration

Kubernetes is thus a container orchestration technology. There are multiple such technologies available today Docker has its own tool called Docker Swarm.

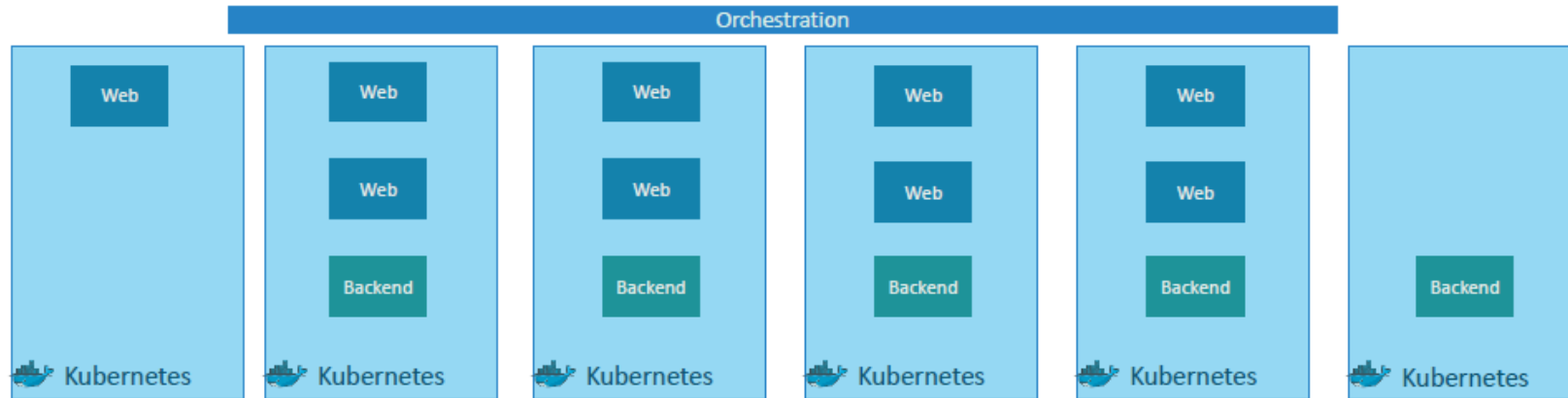
Kubernetes from Google and Mesos from Apache. While Docker Swarm is really easy to setup and get started, it lacks some of the advanced autoscaling features required for complex applications.

Mesos on the other hand is quite difficult to setup and get started, but supports many advanced features. Kubernetes arguably the most popular of it all is a bit difficult to setup and get started but provides a lot of options to customize deployments and supports deployment of complex architectures.

Kubernetes is now supported on all public cloud service providers like GCP, Azure and AWS and the kubernetes project is one of the top ranked projects in Github.



Containers Orchestration



There are various advantages of container orchestration. Your application is now highly available as hardware failures do not bring your application down because you have multiple instances of your application running on different nodes.

The user traffic is load balanced across the various containers. When demand increases, deploy more instances of the application seamlessly and within a matter of second and we have the ability to do that at a service level.

When we run out of hardware resources, scale the number of nodes up/down without having to take down the application. And do all of these easily with a set of declarative object configuration files.

Containers Orchestration

And that is `kubernetes`..

Main Kubernetes Components

Pod

ConfigMap

StatefulSet

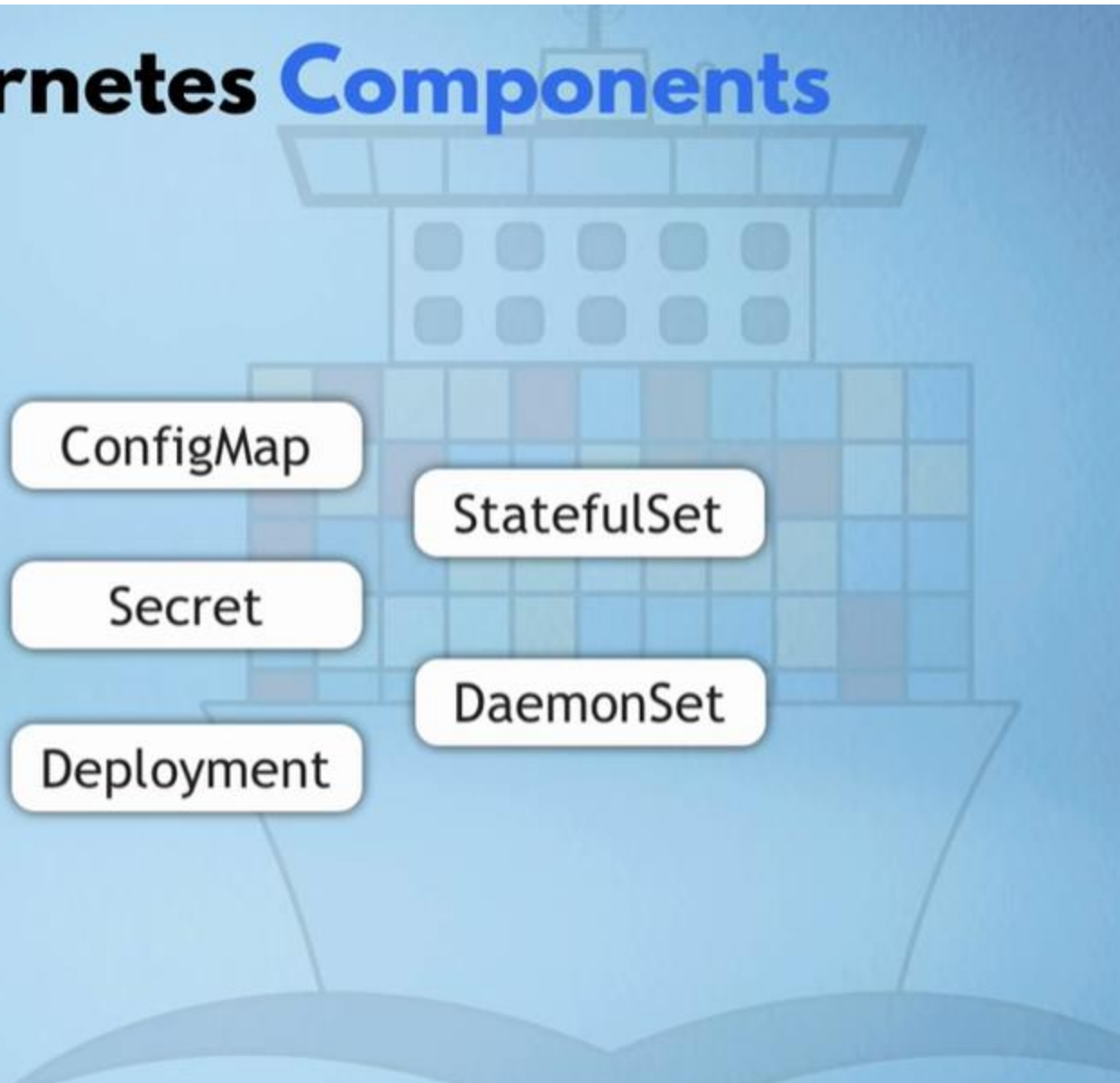
Service

Secret

DaemonSet

Ingress

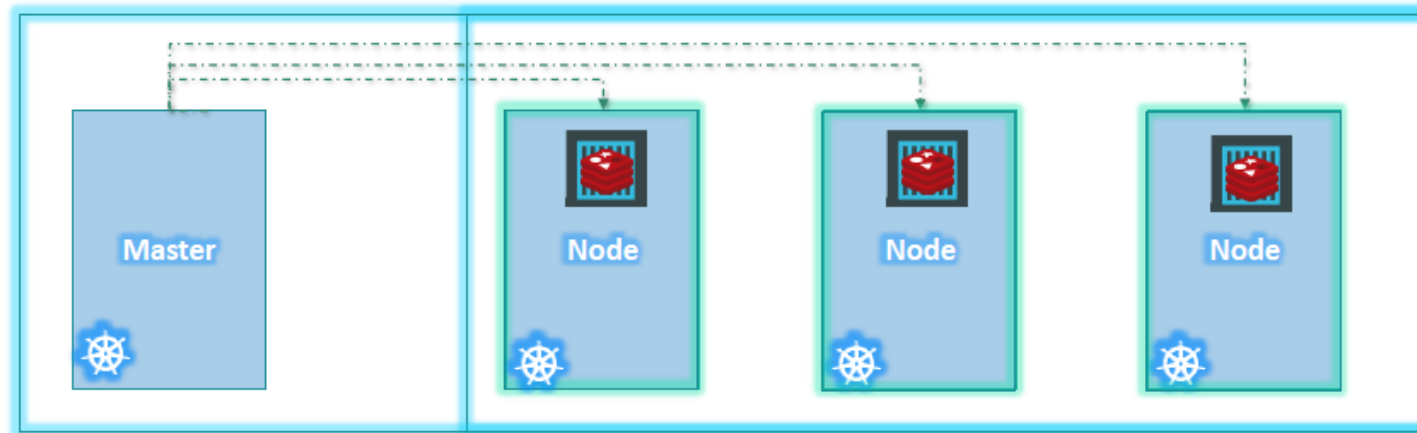
Deployment



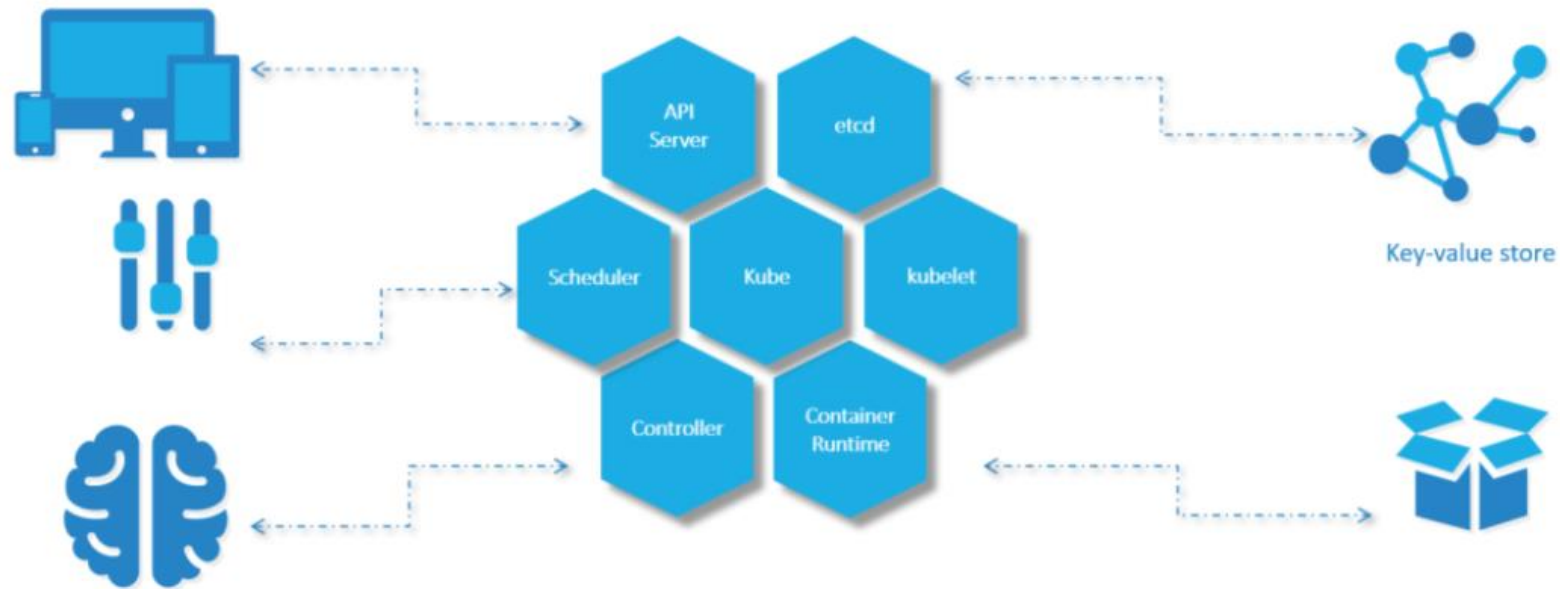
K8d architecture

Now we have a cluster, but who is responsible for managing the cluster? Where is the information about the members of the cluster stored? How are the nodes monitored? When a node fails how do you move the workload of the failed node to another worker node? That's where the Master comes in.

The master is another node with Kubernetes installed in it, and is configured as a Master. The master watches over the nodes in the cluster and is responsible for the actual orchestration of containers on the worker nodes.

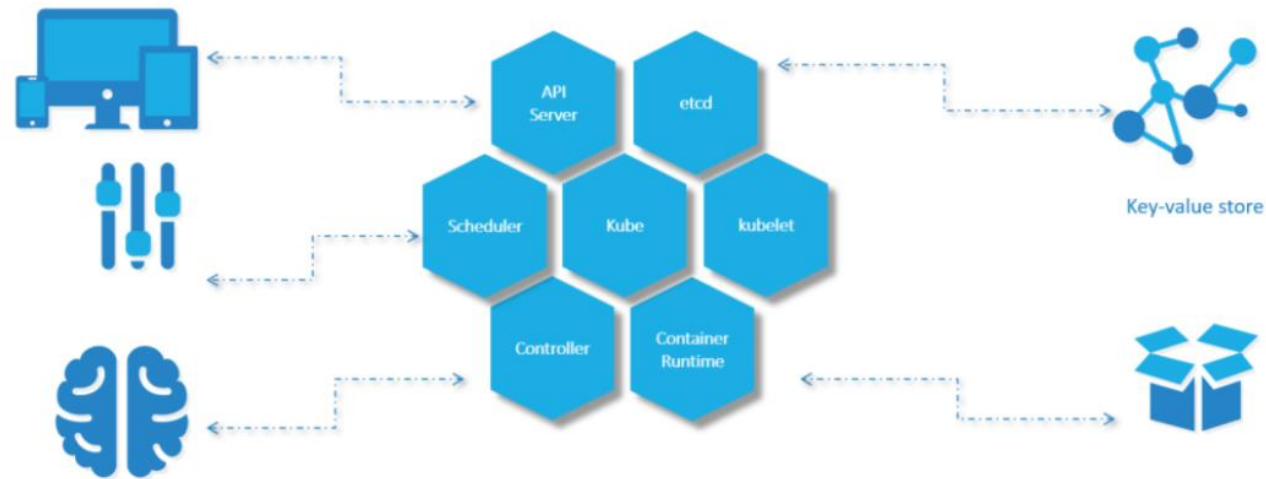


K8d architecture



When you install Kubernetes on a System, you are actually installing the following components. An API Server. An ETCD service. A kubelet service. A Container Runtime, Controllers and Schedulers.

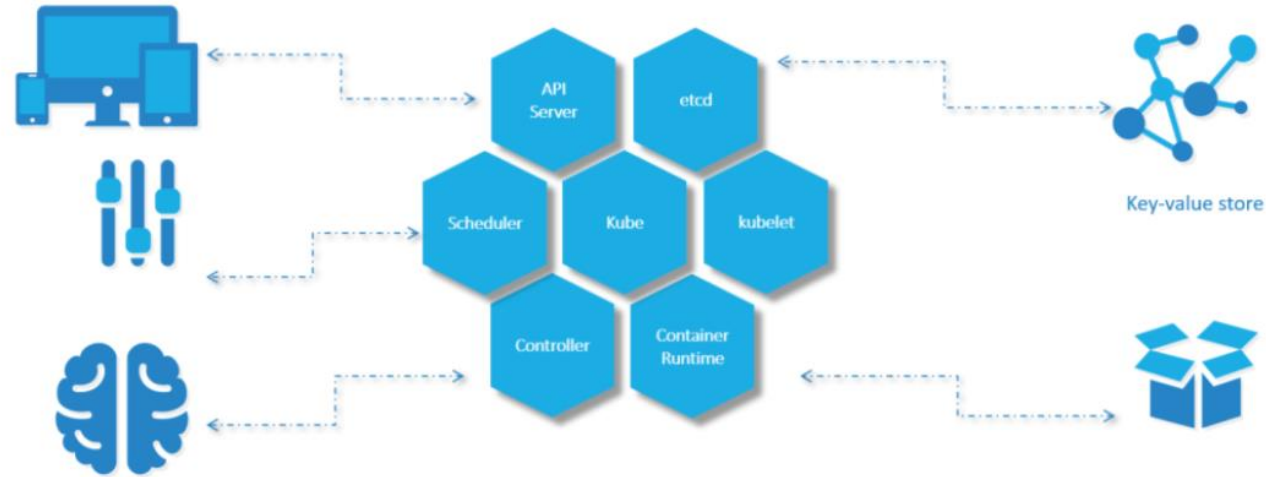
K8d architecture



The API server acts as the front end for kubernetes. The users, management devices, Command line interfaces all talk to the API server to interact with the kubernetes cluster.

Next is the ETCD key store. ETCD is a distributed reliable key value store used by kubernetes to store all data used to manage the cluster. Think of it this way, when you have multiple nodes and multiple masters in your cluster, etcd stores all that information on all the nodes in the cluster in a distributed manner. ETCD is responsible for implementing locks within the cluster to ensure there are no conflicts between the Masters.

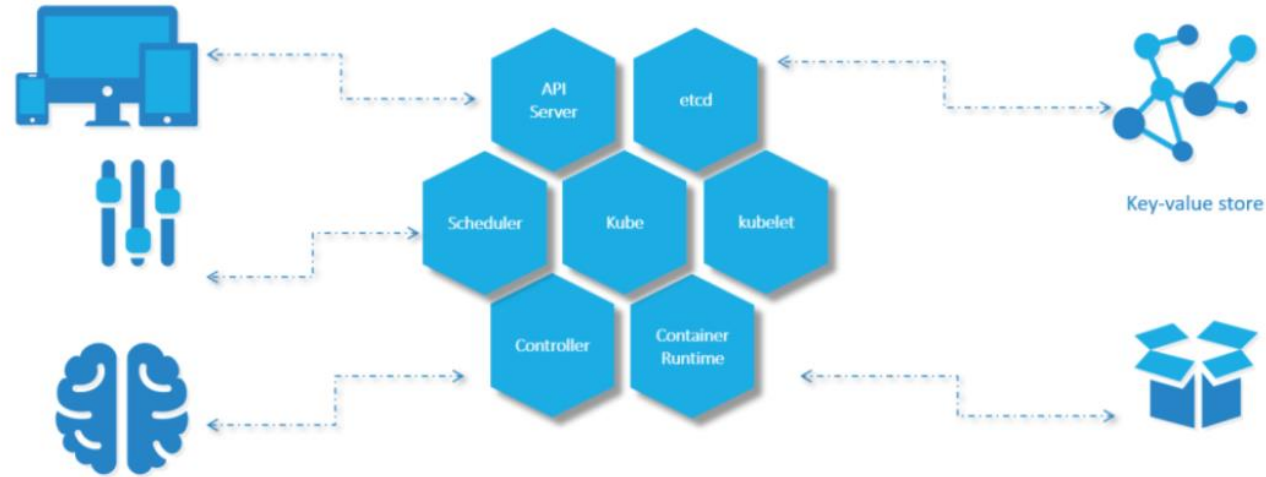
K8d architecture



The scheduler is responsible for distributing work or containers across multiple nodes. It looks for newly created containers and assigns them to Nodes.

The controllers are the brain behind orchestration. They are responsible for noticing and responding when nodes, containers or endpoints goes down. The controllers makes decisions to bring up new containers in such cases.

K8d architecture



The container runtime is the underlying software that is used to run containers. In our case it happens to be Docker.

And finally kubelet is the agent that runs on each node in the cluster. The agent is responsible for making sure that the containers are running on the nodes as expected.

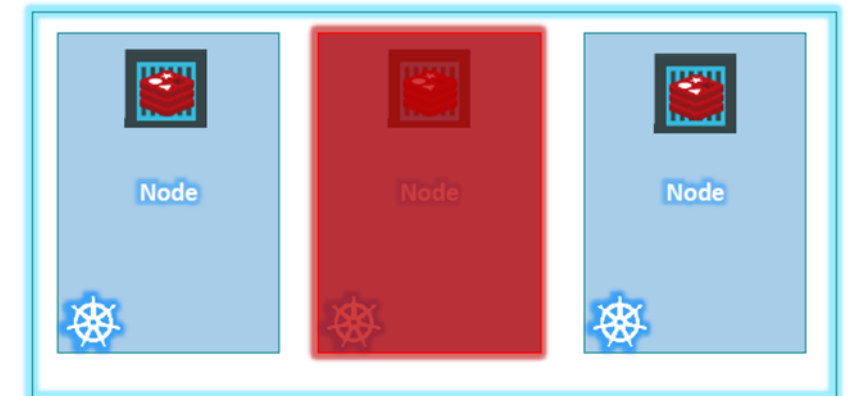
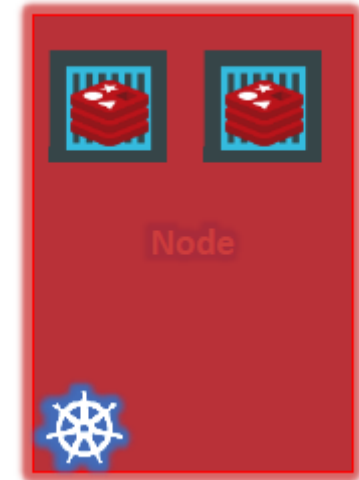
K8s architecture

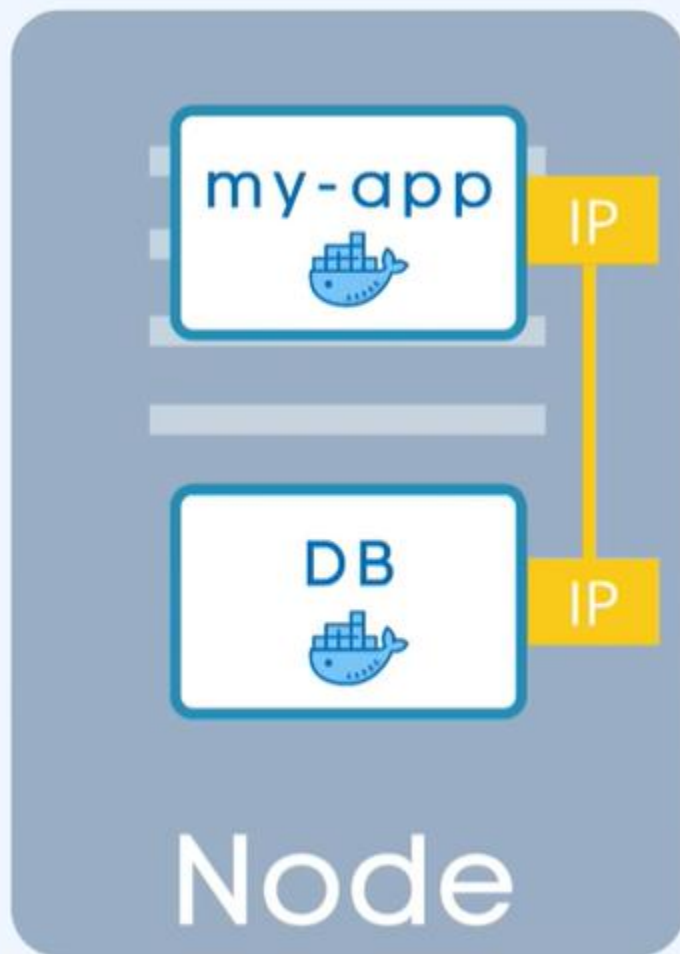
A node is a machine physical or virtual on which kubernetes is installed. A node is a worker machine and this is where containers will be launched by kubernetes.

It was also known as Minions in the past. So you might hear these terms used interchangeably.

But what if the node on which our application is running fails? Well, obviously our application goes down. So you need to have more than one nodes.

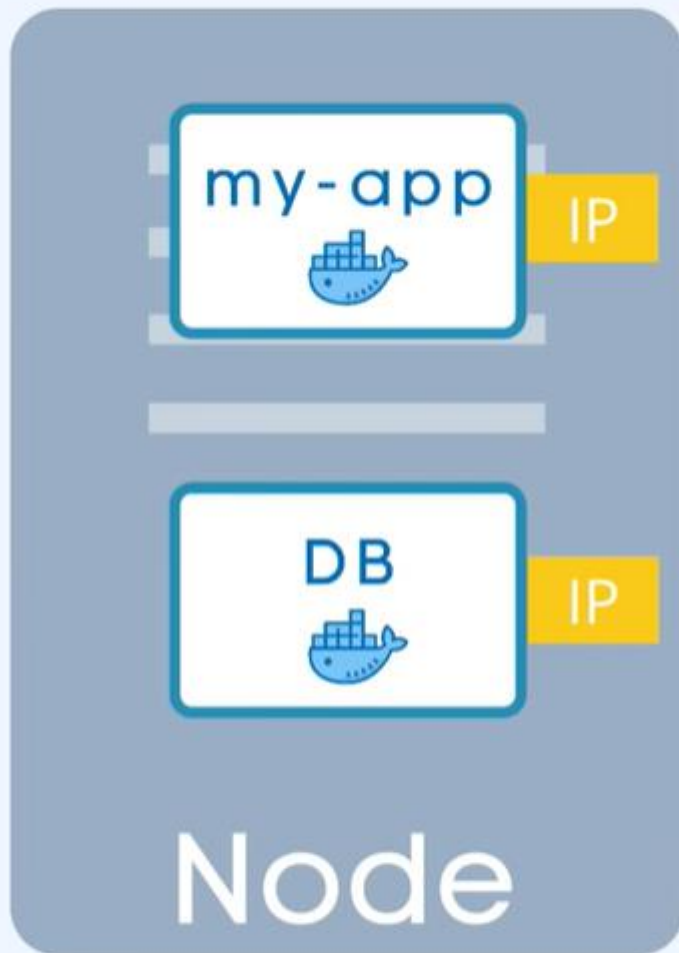
A cluster is a set of nodes grouped together. This way even if one node fails you have your application still accessible from the other nodes. Moreover having multiple nodes helps in sharing load as well.





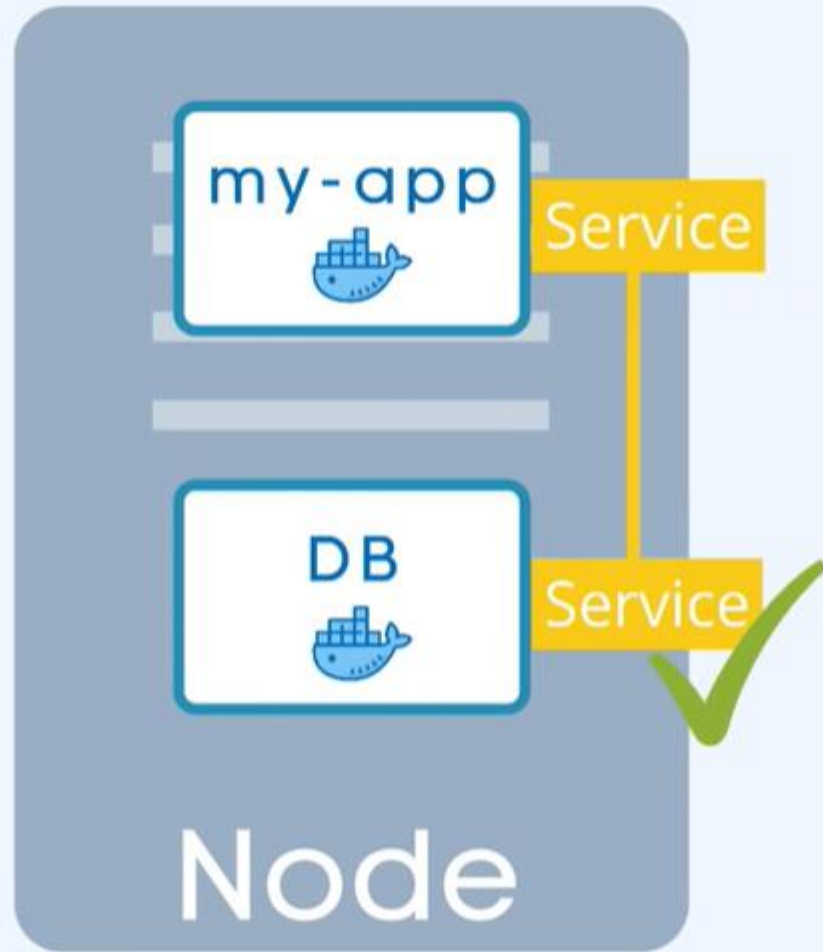
- ▶ **Smallest** unit in Kubernetes
- ▶ **Abstraction** over container
- ▶ Usually **1 Application** per Pod
- ▶ Each Pod gets its **own IP address**

Pods are **ephemeral**!



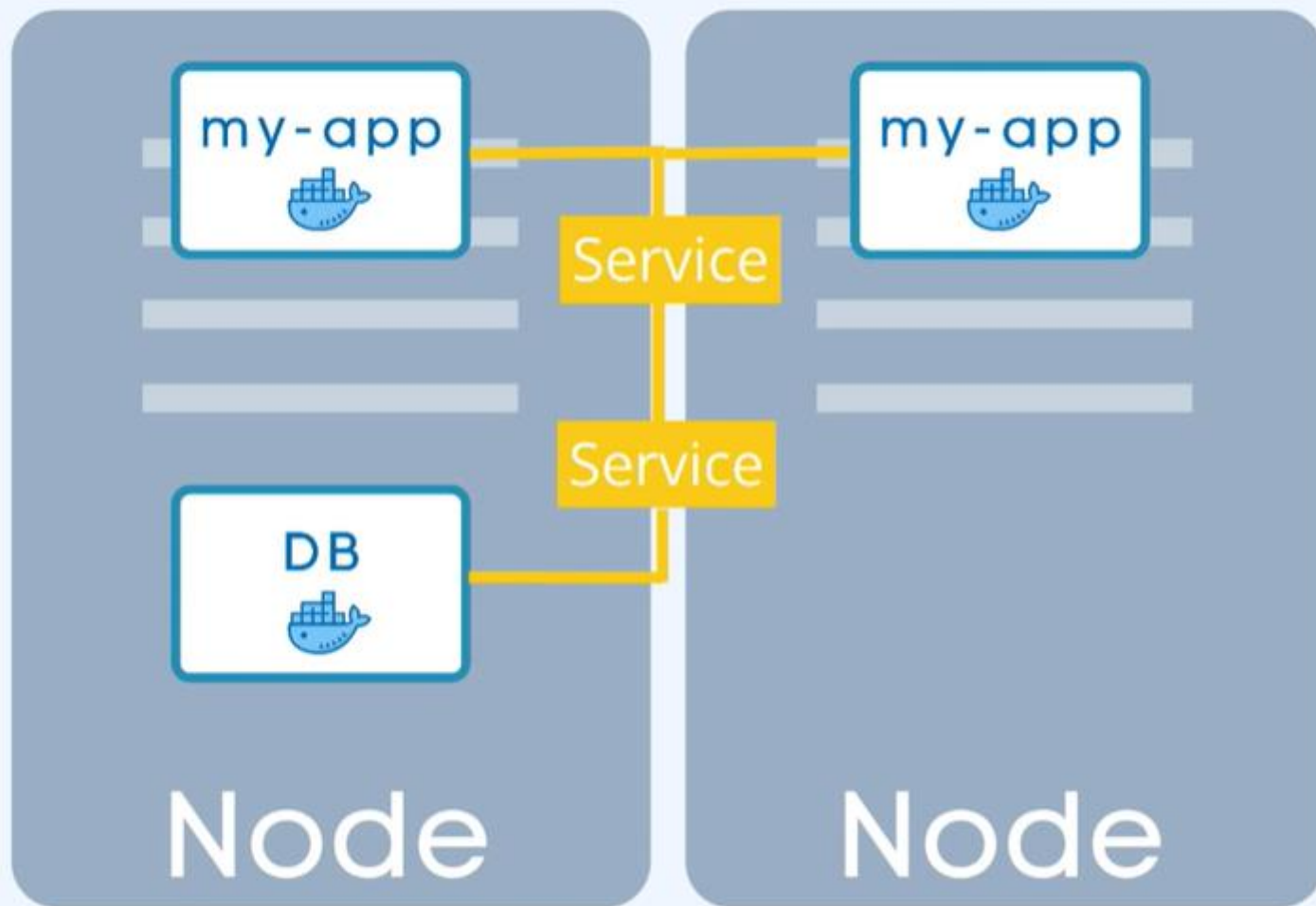
- ▶ **Smallest** unit in Kubernetes
- ▶ **Abstraction** over container
- ▶ Usually **1 Application** per Pod
- ▶ Each Pod gets its **own IP address**
- ▶ **New IP address** on re-creation





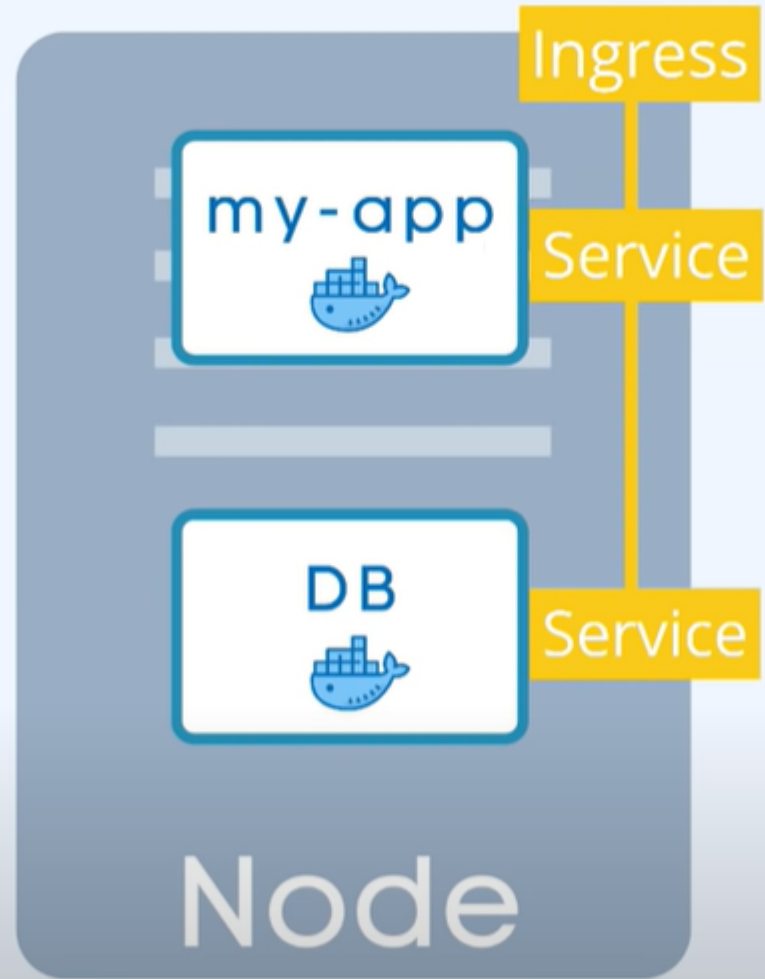
- ▶ **Permanent** IP address
- ▶ Lifecycle of Pod and Service **not connected**





► permanent IP

► load balancer



K8d architecture

Services

Kubernetes services enable communication and load balancing between different sets of pods, making applications discoverable and resilient.

Service Types:

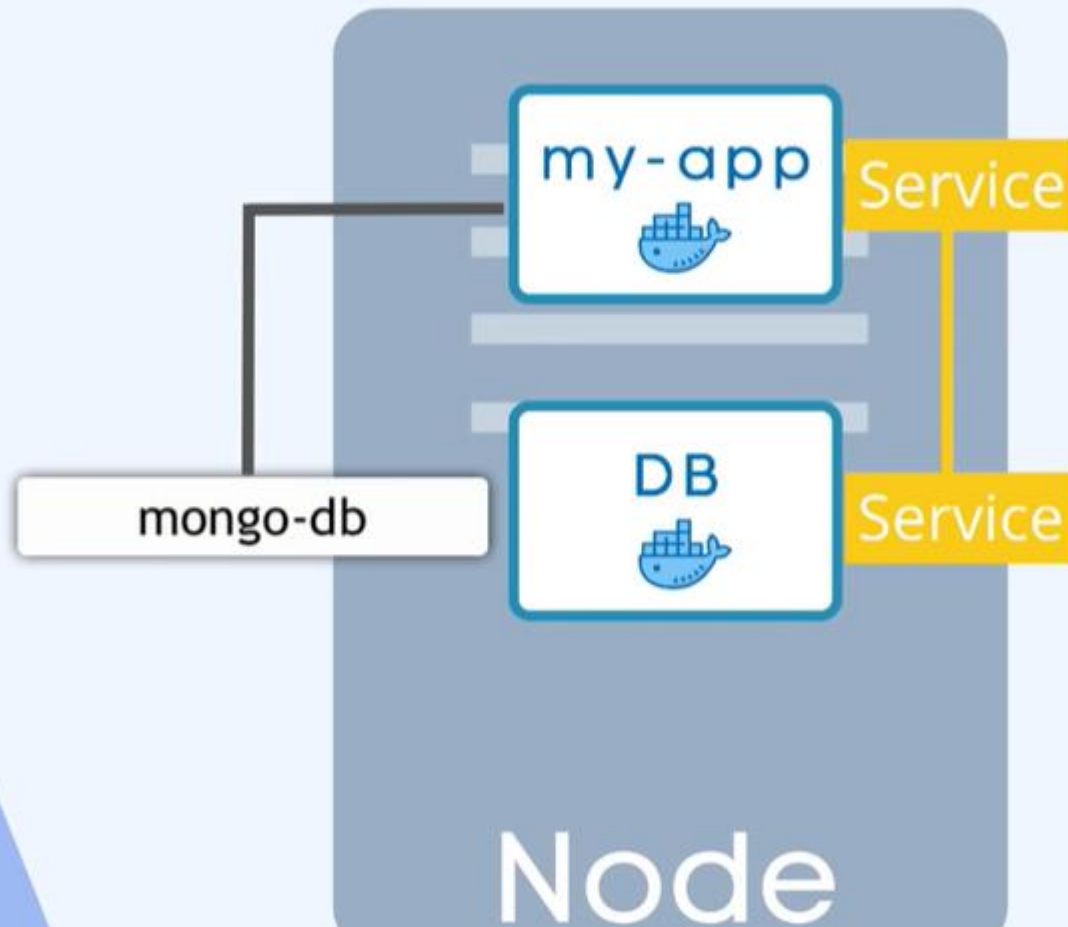
- ClusterIP: Exposes the service within the cluster for internal communication.
- NodePort: Exposes the service on a static port on each node, suitable for development/testing.
- LoadBalancer: Creates an external load balancer for internet or external network access.
- ExternalName: Maps the service to a DNS name, enabling access to external services.

Ingress

Ingress manages external access to services in the cluster, providing routing for HTTP and HTTPS traffic without modifying application code.

Ingress Controller: Watches for changes to ingress resources, configuring infrastructure to implement defined routing rules.

Database URL usually in the
built application!



Re-build

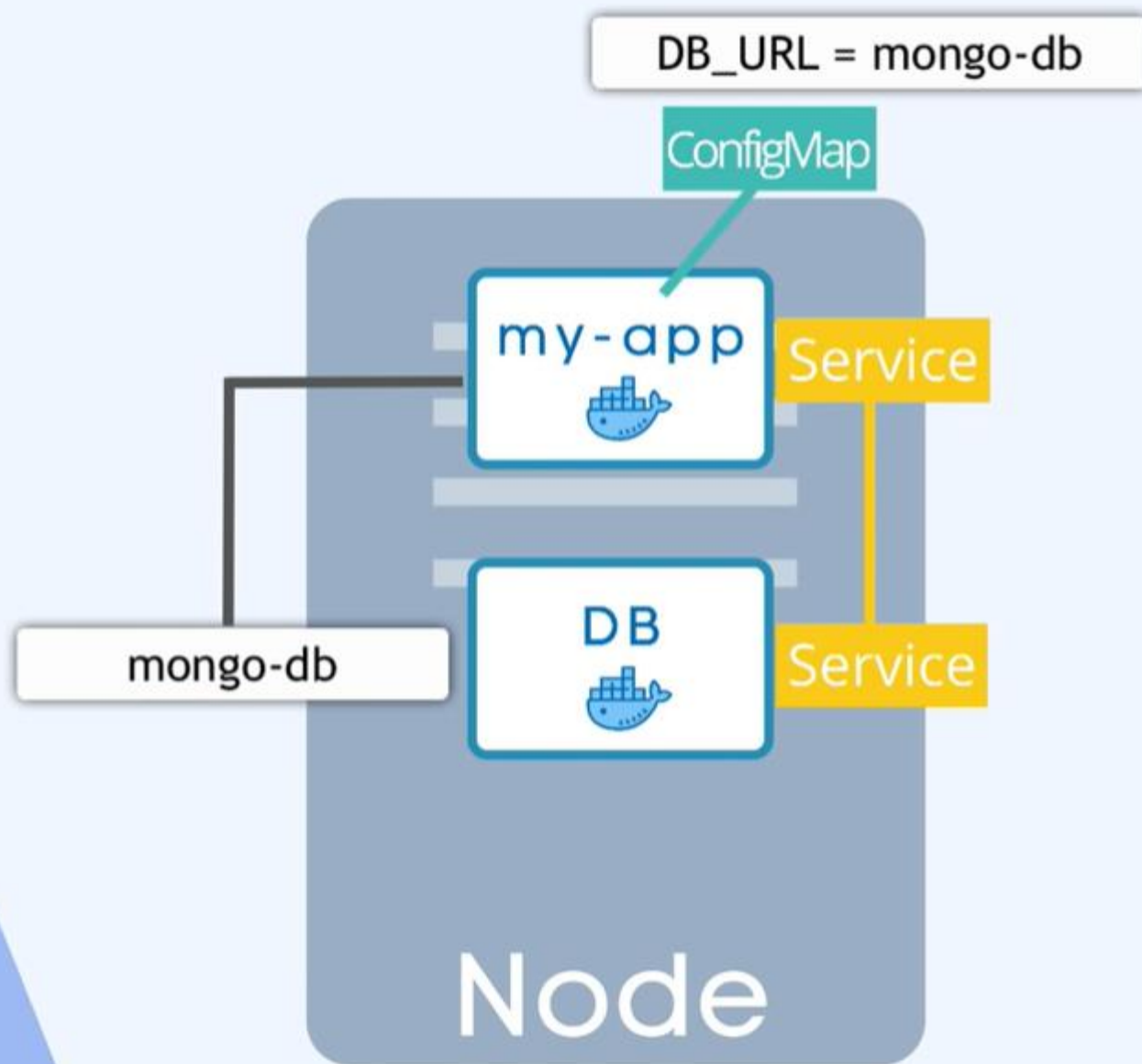


Push it to repo

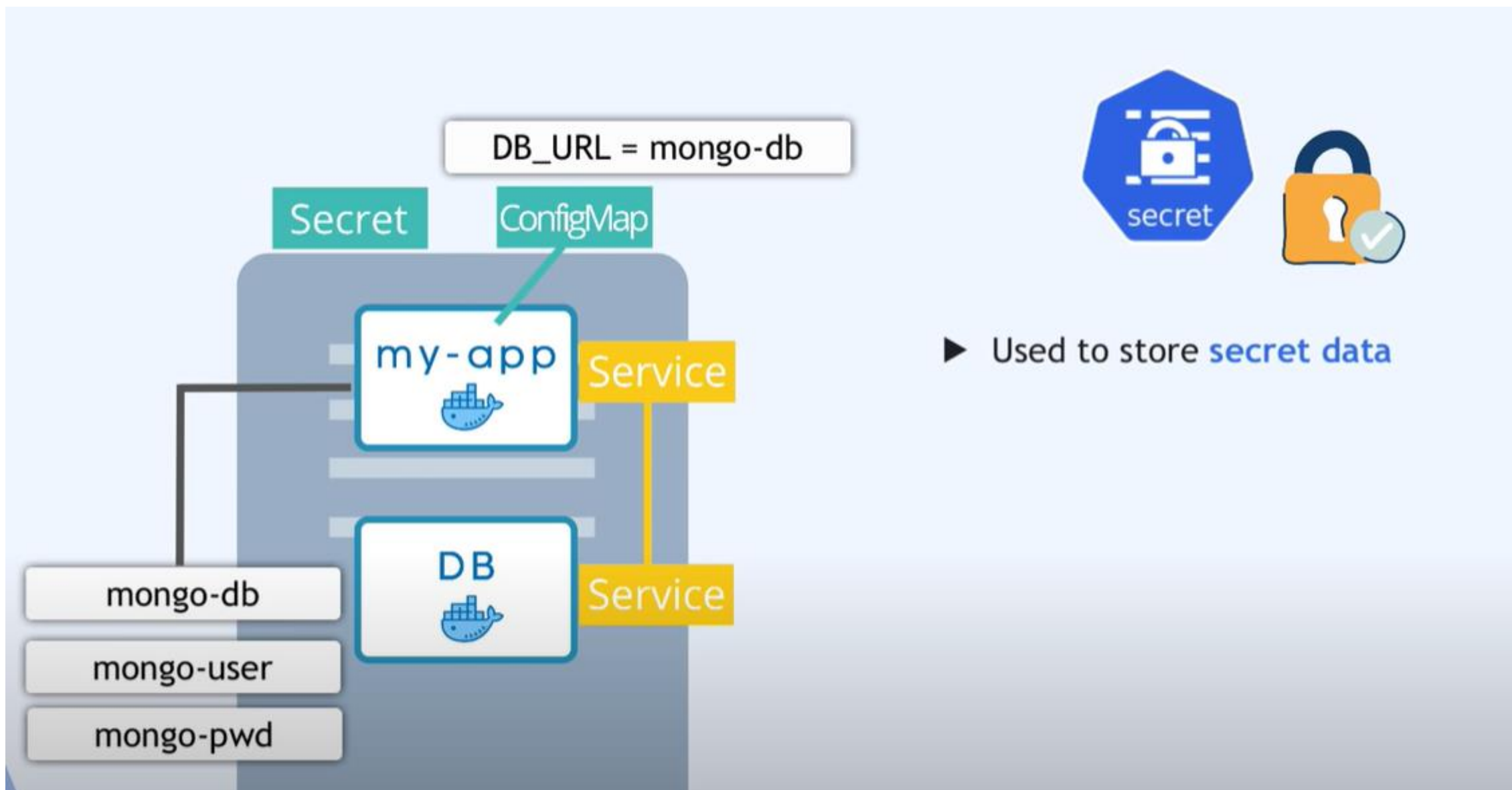


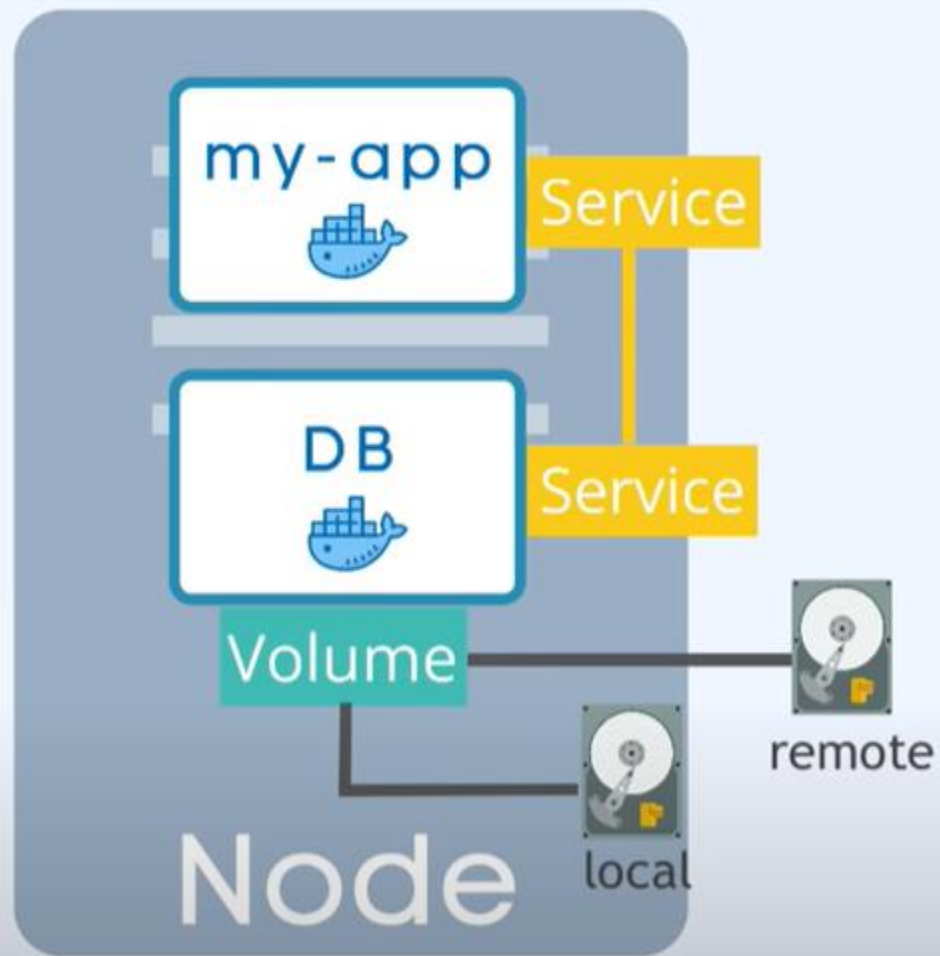
Pull it in your Pod



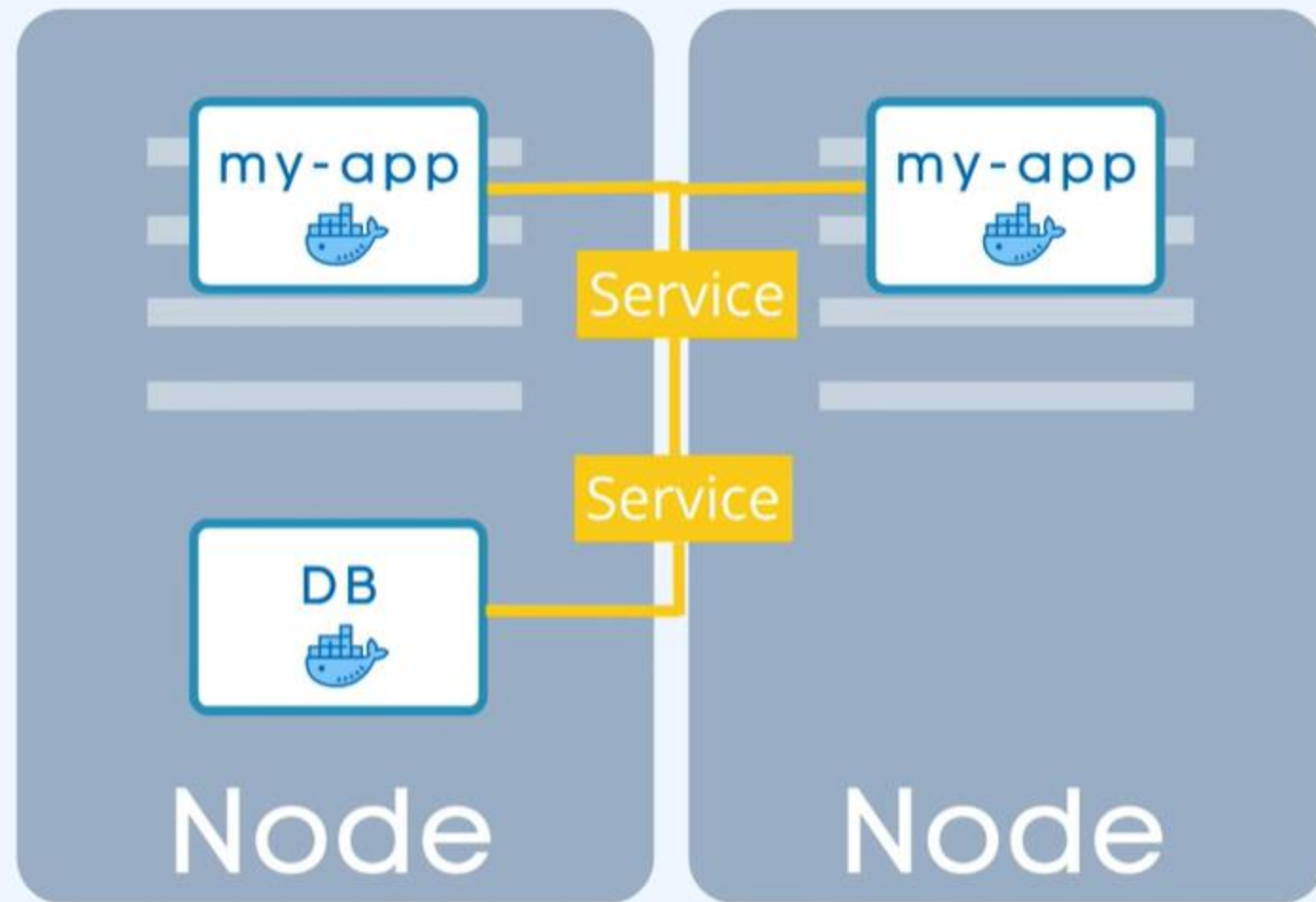


- **External** Configuration of your application

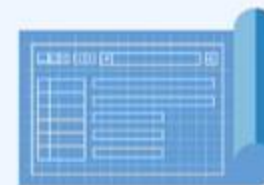




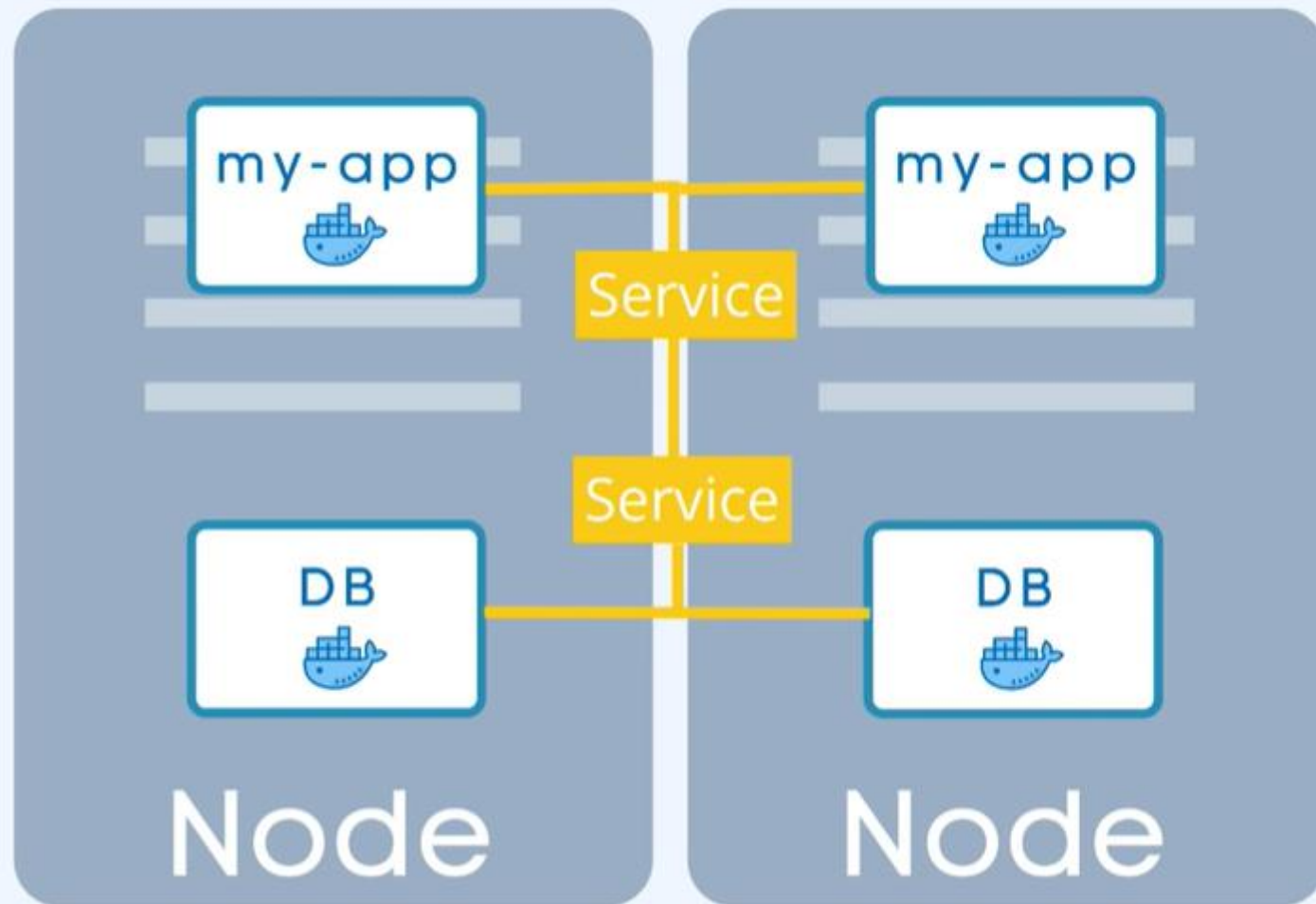
- ▶ Storage on **local** machine
- ▶ Or **remote**, outside of the K8s cluster



DEPLOYMENT



- ▶ **Blueprint** for "my-app" Pods
- ▶ You create **Deployments**



Deployment =
for stateLESS Apps

StatefulSet =
for stateFUL Apps or Databases

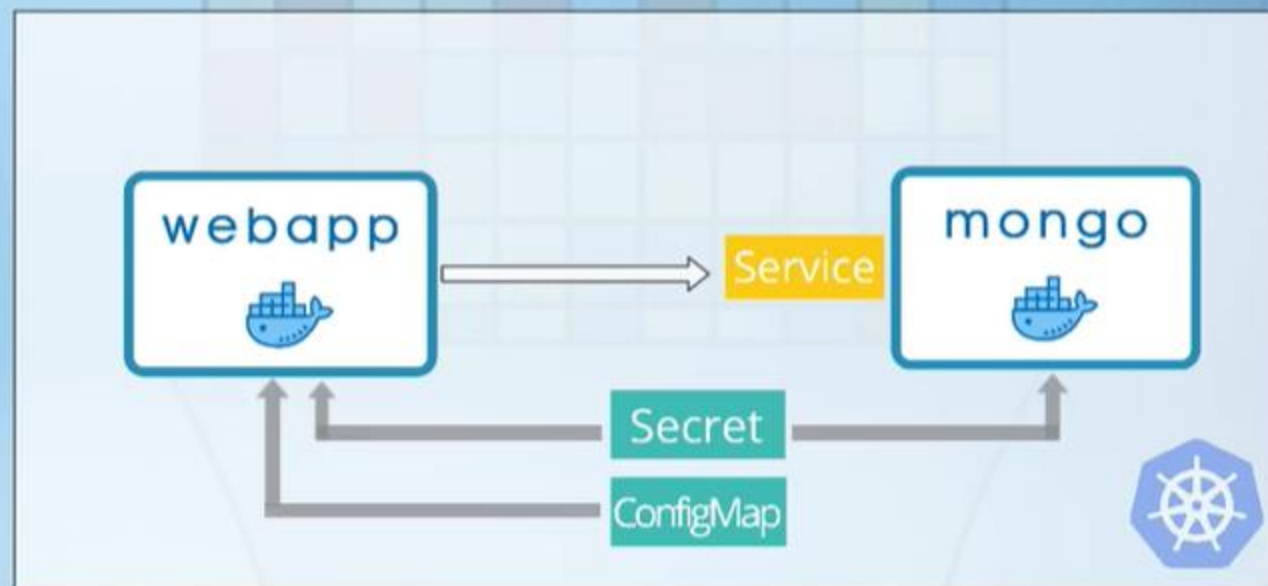
Demo Project Overview

Deploy
MongoDB Database

Deploy
Web Application



External Configuration
for DB URL and Credentials



K8s Components Overview

► Create 4 K8s Config Files



ConfigMap

MongoDB Endpoint

Secret

MongoDB User & Pwd

Deployment

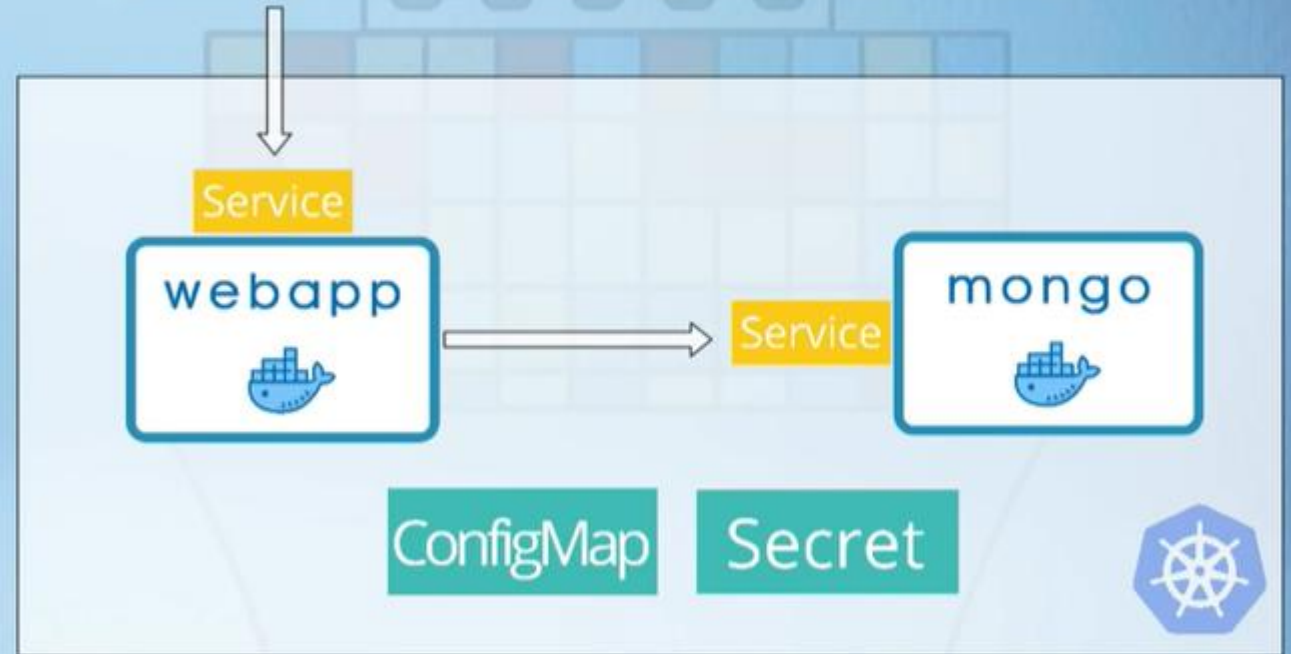
MongoDB Application with internal Service

Service

Deployment

Our own WebApp with external Service

Service



ConfigMap

Create a configMap file mongo-config.yaml

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
```

```
  name: mongo-config
```

```
data:
```

```
  mongo-url: mongo-service
```

Name of the configmap

Key/value

Secret

Create a secret file named mongo-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-secret
type: Opaque
data:
  mongo-user: bW9uZ291c2Vy
  mongo-password: bW9uZ29wYXNzd29yZA==
```

Key/value

A secret should be Base64 encrypted

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  labels:
    app: mongo
```

```
spec:
  replicas: 1
```

Number of replicas

```
selector:
  matchLabels:
    app: mongo
```

```
template:
  metadata:
    labels:
      app: mongo
```

Template of the deployed container

```
spec:
  containers:
    - name: mongodb
      image: mongo:5.0
      ports:
        - containerPort: 27017
```

Specs of the container

```
env:
  - name: MONGO_INITDB_ROOT_USERNAME
    valueFrom:
      secretKeyRef:
        name: mongo-secret
        key: mongo-user
  - name: MONGO_INITDB_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mongo-secret
        key: mongo-password
---
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  labels:
    app: mongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongodb
          image: mongo:5.0
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-user
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-password
---
```

→ All replicas share the same label but each has a unique name

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  labels:
    app: mongo
```

Belong to this deployment

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
```

All pods that match this label

```
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongodb
          image: mongo:5.0
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-user
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-password
```

Environment variables provided by Mongo image for username and password. They are attached to the secret values already defined in secret file

```
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  selector:
    app: mongo
  ports:
    - protocol: TCP
      port: 27017
      targetPort: 27017
```



Attach it to the app


```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webapp-deployment
5    labels:
6      app: webapp
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: webapp
12    template:
13     metadata:
14       labels:
15         app: webapp
16     spec:
17       containers:
18         - name: webapp
19           image: nanajanashia/k8s-demo-app:v1.0
20           ports:
21             - containerPort: 3000
22           env:
23             - name: USER_NAME
24               valueFrom:
25                 secretKeyRef:
26                   name: mongo-secret
27                   key: mongo-user
28             - name: USER_PWD
29               valueFrom:
30                 secretKeyRef:
31                   name: mongo-secret
32                   key: mongo-password
33             - name: DB_URL
34               valueFrom:
35                 configMapKeyRef:
36                   name: mongo-config
37                   key: mongo-url
38  ---

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort
  selector:
    app: webapp
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
      nodePort: 30100

```

Expose to outside

Outside port

Mainly it will not work on Windows 😊

- `Kubectl apply -f mongo-config.yaml`
 - `Kubectl apply -f mongo-secret.yaml`
 - `Kubectl apply -f mongo.yaml`
 - `Kubectl apply -f webapp.yaml`
 - `Kubectl get all`
 - `Kubectl get configmap`
 - `Kubectl get secret`
 - Minikube dashboard
 - `minikube start`
 - `minikube status`
 - `kubectl get nodes`
-
- `kubectl expose deployment webapp-deployment --type=LoadBalancer --port=3000`
 - `kubectl get services`
-
- `minikube service webapp-deployment`